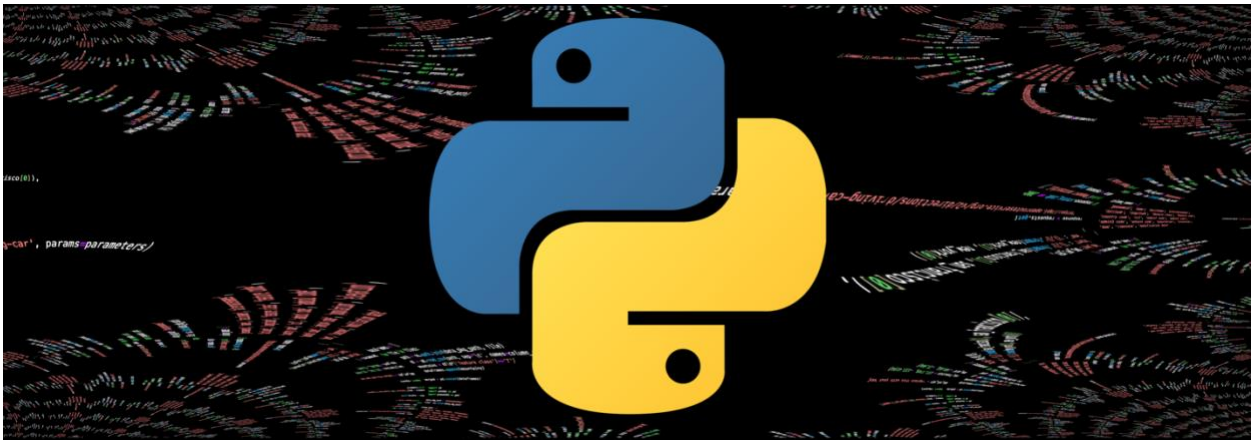
SDS 210 course information

SDS 210 - Programming with Spatial Data, 5 ECTS, Module Coordinator: Hendrik Wulf

Course website: SDS 210 - Jupyter Book, MS Teams class: 26FS SDS210

Labs: Thursdays 16:15 – 18:00, Fridays 10:15 – 12:00, and 12:15 – 13:45 in Y25-J-09/10



1. Scope & objectives

1.1 Description

Welcome to SDS210, ESS342 and GEO876: Programming with Spatial Data. This course focuses on computational thinking and Python programming for geospatial applications. In an increasingly data-driven and automated world, the ability to design, write and organise code for spatial analysis is an essential skill for addressing environmental, social and urban challenges. Through a hands-on and problem-based learning approach, this course introduces the tools and techniques needed to programmatically understand, analyse, and visualise spatial data. You will develop a strong foundation in Python programming, progressively applying it with key geospatial libraries such as GeoPandas, RasterIO and Matplotlib. The course emphasises writing modular, well-documented and reproducible code, supported by Jupyter Notebooks and Git to enable transparent and collaborative workflows. The course concludes with an individual programming project in which you will design and implement your own solution to a spatial data problem. Upon completion of SDS210, you will be well-prepared to take on more advanced spatial data science challenges in research and industry.

1.2 Competencies

In this course you can acquire foundational programming skills and practical expertise in spatial data analysis. On completion of this course, you will be able to:

- **Apply** core Python programming concepts, including variables, loops, functions, and data structures.

- **Develop** a conceptual outline and workflow before implementing a coding solution.
- **Write** well-documented, modular, and reproducible Python code following best practices.
- **Design and implement** object-oriented programming solutions that can read, store, manipulate, visualize, and export spatial data.

1.3 ECTS

Successful completion of the course will earn you **5 ECTS** credits.
Based on the rule that one ECTS credit point equals 30 working hours, the average time per student for this course should be 150 hours in total. The table below indicates the average time a student should expect to spend per lecture and lab exercise.

Type	Times	Content	Hours (h)	Sum
Flipped-classroom	12	3 h script & exercises + 1 h independend learning	12 x ~4 h	~50
Lab sessions	12	2 h preparation + 2 h practicals + 1 h revision	12 x ~5 h	~60
Individual project	1	Project report and presentation	1 x ~40 h	~40
				150

Please note that we schedule **seven out of nine weekly hours for self-study** outside the labs. We **highly recommend** spending this time reviewing the content of each session to ensure continuous learning and progress. This workload distribution reflects the course's emphasis on **continuous practice** and independent problem solving, rather than exam preparation.

2. Course structure

2.1 Program

W	L	Thu	Fri	Topic	Project
1	1	19.02	20.02	Get Started with Python	
2	2	26.02	27.02	Variables & Values	
3	3	05.03	06.03	Loops and Conditional Statements	
4	4	12.03	13.03	Understanding and Defining Functions	
5	5	19.03	20.03	Handling Data, Dates and Text	
6	6	26.03	27.03	Using Libraries, Web APIs & Git	
7	7	02.04	03.04	Working with NumPy & Co.	
8	-	09.04	10.04	Easter Break	
9	8	16.04	17.04	Working with Pandas & Co.	Introduction
10	9	23.04	24.04	Working with Matplotlib & Co.	
11	10	30.04	01.05	Object-Oriented Programming	
12	11	07.05	08.05	Best Practices	
13	-	14.05	15.05	Project Feedback	
14	-	21.05	22.05	Project Feedback	Report
15	-	28.05	29.05	Project Presentations	Presentation

W = Week. L = Lesson. Dates in red mark holidays.

2.2 Time & Space

Labs times: Thursdays: 16:15-18:00; Fridays: 10:15-12:00 and 12:15-13:45

Labs rooms: Y25-J-09/10 (should the others be full, you can also use room Y25-J-08)

Assessment: Project report (50%) + 10-minutes individual project presentation in week 14 (50%)

2.3 Flipped Classroom Concept

The course follows a flipped classroom approach that blends self-paced learning with interactive lab sessions. Each week, you will engage with the script (website) and accompanying exercises and practicals **before** the lab sessions. These materials introduce the core programming concepts and spatial data techniques for that week.

The in-person lab sessions are then used to **apply** what you have learned, **clarify** open questions, and work collaboratively on practical exercises. Instructors will be available to guide you, discuss your code, and help troubleshoot challenges you encounter while implementing your solutions.

This approach allows you to:

- Learn theoretical concepts at your own pace through focused content on the course website.

- Deepen understanding during labs through hands-on coding and peer exchange.
- Receive targeted support where you need it.

All required materials (scripts, exercises, videos, and practicals) are available on the **course website**. You are encouraged to review the content regularly and make active use of the labs to consolidate your understanding. Asking questions early and sharing insights with peers are key to success in this course.

Before each lab, you are expected to

- read the script of the lesson,
- do the accompanying exercises of each session and
- attempt the practicals.

Labs are not introductory lectures; they are problem-solving sessions that assume preparation.

2.4 Language

All course materials (including scripts, exercises, practicals, videos and resources) are provided in **English**. English is also the primary language used in written project reports and assessments to ensure consistency for all students and accessibility for instructors.

2.5 Attendance

Regular participation in the lab sessions is **strongly recommended** to successfully complete this course. The labs form an integral part of the flipped-classroom approach, providing an opportunity to apply your learning, receive personalized feedback and discuss challenges with instructors and peers.
Programming is a skill that is best developed through **consistent practice and interaction**. Active engagement in the labs will help you to deepen your understanding, avoid common pitfalls and keep up with the weekly exercises and project work.

3. Assessment

The course assessment is based on an **individual programming project** that integrates the concepts, methods, and coding practices learned throughout the course. This project represents an opportunity to demonstrate your ability to design, implement, and communicate a reproducible solution to a spatial data problem using Python. The final mark will be determined by both parts (weighted average), the **report (50%)** and the **presentation (50%)**.

3.1 Programming Projects

Around mid-semester, when entering lesson 8 *"Working with Pandas & Co."*, you will be able to choose one from several predefined project topics. Each project will involve open data and a real-world spatial question. You will design and implement your own code-based solution using Python and relevant geospatial libraries (e.g., *GeoPandas*, *RasterIO*). You are strongly encouraged to start exploring project

topics by week 8, sketch a workflow by week 10, and iteratively refine your solution during the remaining lab sessions.

3.2 Project reports

Each project is accompanied by a **short written report (max. two pages, in pdf-format)** that documents:

- your conceptual approach and workflow,
- the key steps of your implementation,
- challenges encountered and how you addressed them, and
- a reflection on your use of AI tools, if applicable.

Your report must include a link to your public GitHub (or GitLab) repository. Your repository must allow another student to reproduce your results on a different machine. This includes clear instructions, consistent environments, relative file paths, and executable notebooks from top to bottom. This repository should therefore contain a README file that documents your environment and setup. **Your Python notebook (.ipynb)** should include well-documented code, functions, methods and classes, as well as descriptive comments and meaningful variable names. For project submission, attach a Markdown export of your notebook in pdf-format to your short written report (2-pages).

You have to submit your project report via the respective MS Teams (see section 5.1 below) assignment. The assignment will be published on Monday, 13 April 2026. The deadline for the **project report submission** is **22 May 2026 (Friday) at 18:00**. Please refer to the rubric in the assignment for further details of the assessment criteria:

Projects will be evaluated based on the following criteria:

- Correctness: The code runs from start to finish with no logical errors.
- Robustness: It handles errors and edge cases gracefully.
- Structure: Effective use of functions and/or classes; modular design; no unnecessary duplication.
- Readability: Clear syntax; consistent formatting and indentation; meaningful names.
- Efficiency: Avoids redundant work and uses appropriate data structures and methods.
- Documentation: Concise docstrings and comments that explain parameters and return values.
- Reproducibility: A peer can rerun the project on a different machine
- Report: Provides a clear summary of the problem, approach, results and challenges.
- Innovation: Original or thoughtful ideas in design, analysis or visualisation.
- Explanation & demo: Justify design choices and demonstrate all functionality.

3.3 Project presentations

At the end of the semester, you will give a **10-minute individual presentation** where you:

- explain your problem definition and modelling approach,
- demonstrate key parts of your code, and
- justify your design decisions and findings.

This discussion ensures that you understand and can explain your code, and provides a chance for personalised feedback. Please bring your laptop with the running code to the discussion.

Repeatability
If the final project presentation is not passed, it can be retaken once end of August.

4. Course materials

All course materials are provided through the course website: https://hendrikwulf.github.io/sds210-jb/

The website serves as the central reference for the course and includes:

- software installation and setup instructions
- weekly practical exercises and lab materials
- programming examples and explanations
- project descriptions and submission guidelines (mid semester)

You are expected to consult the course website regularly, as it is continuously updated throughout the semester and reflects the most current version of the course content. In other words, it is currenlty under construction and will see updates throughout the semester.

5. Communication

5.1 MS Teams

MS Teams is the central communication platform for this course. It is used to:

- share announcements and course information
- ask and answer questions
- submit the final project report

All participants **must join** the MS Teams team "26FS SDS210 – Programming with Spatial Data" at the start, using the access code: **30gyzuu**.

To ensure consistent communication, OLAT is used only to provide this course syllabus. All other course-related communication takes place on MS Teams.

If you need help outside of lab hours, please post your question in the appropriate lab channel on Teams. Other students are encouraged to respond and help where possible. This shared discussion space benefits everyone and reduces duplicate questions.

5.2 Contact

Questions about course content or exercises

For questions related to the course material (e.g. *"How do I resolve problem X in task Y?"*), please use the respective Microsoft Teams channel. This helps everyone benefit from the discussion.

To help us support you efficiently, always include:

- a brief description of the problem
- the exact error message (if any)
- a screenshot or code snippet
- a Colab or GitHub link (when applicable)

Personal or administrative questions

For personal matters or course-related questions specific to SDS210, please contact the module coordinator: hendrik.wulf@geo.uzh.ch

Office hours (during the semester): Tuesdays and Wednesdays, 14:00–15:00 in room Y25-K-12

6. Use of AI & Integrity

In this course, we encourage the purposeful, transparent, and reflective use of AI tools as part of your learning process. AI tools may be used during lab sessions as learning aids, like online documentation, forums, or peer discussion, provided their use is transparent and reflective. When used responsibly, AI can assist with tasks such as explaining complex topics, improving grammar and readability, reviewing code, exploring alternative solutions and receiving feedback on ideas. When used in this way, AI can be a valuable learning aid.

The primary goal of this course, however, is for you to *develop your own knowledge, skills, and competencies* in spatial programming. AI tools may support this process, but they must not replace your own thinking, understanding, or decision-making.

Your responsibility

- You are responsible for all content submitted for the project report.
- Any AI-assisted content must be critically reviewed, edited, and validated by you.
- You must clearly state when and how AI tools were used.
- Simply copying AI-generated text or code without revision or acknowledgment is considered plagiarism and will be sanctioned.

What counts as plagiarism

Plagiarism is defined as presenting the work of others (whether from AI tools, websites, publications, or other external sources) as your own.

Example disclosure statement

An appropriate statement describing the use of AI at the end of the project report might read:

"I used a large language model as an assistance tool during the preparation of this project report to:

- *check and correct grammar and spelling,*
- *improve linguistic clarity and coherence,*
- *debug and reflect on code for data processing and analysis.*

Apart from the uses listed above, no AI tools were used in the preparation of this submission."

Institutional guidelines

When using AI tools, you must comply with all applicable regulations and guidelines of the University of Zurich and the Faculty of Science. If you are unsure whether a specific use of AI is permitted, ask the course instructors before submitting your work.